Semanas 7 y 8 · Paso a paso detallado

Versión ampliada de las guías de las semanas 7 y 8, sin depender de la consola serie del ESP32.

Antes de empezar (vale para las dos semanas)

Paso 0.1. Qué ventanas vas a usar

Solo necesitas **una ventana con SSH a la Raspberry** (la llamaremos **Ventana A**). En la laptop: tecla Windows → escribe **PowerShell** → Enter, y luego:

```
ssh usuariok@integradoraPi4.local
```

Escribe la contraseña (no se ven las letras; es normal). Estás dentro cuando aparece `usuariok@integradoraPi4:~ $`. Activa el entorno del proyecto:

```
cd ~/proyecto-kyber-rpi/pi && source ../src/venv/bin/activate
```

Debe aparecer `(venv)` al inicio de la línea. **Repite este comando cada vez que abras una ventana nueva.**

> **No necesitas la consola serie (miniterm)**
>
> Todo lo que hace el ESP32 llega a la Raspberry dentro del resultado JSON: `esp_estado`, `esp_t_hs_us`, `esp_heap_usado_hs` y `esp_stack_libre`. Por eso estas dos semanas se hacen sin el miniterm. Que el miniterm se quede «cargando» sin mostrar nada no es un error: el ESP32 imprime sus mensajes solo al arrancar, y si lo abres después no ves nada.
>
> Si alguna vez necesitas ver lo que imprime el ESP32 (solo para diagnosticar un fallo), usa la laptop: pasa el USB del ESP32 a la laptop, abre la terminal de ESP-IDF y ejecuta `idf.py -p COM3 monitor` (con tu puerto). Presiona el botón **EN** y verás el arranque. Sales con **Ctrl+]**. Luego regresa el USB a la Raspberry.

Paso 0.2. Comprobar que todo sigue como en la semana 6

Con el ESP32 conectado por USB a la Raspberry, en la Ventana A:

```
cd ~/proyecto-kyber-rpi/pi && source ../src/venv/bin/activate
python3 servidor_tls.py --grupo clasico --frame 128
```

Debe terminar con `"esp_estado": 0` y sin la línea `"error"`. Si es así, el montaje, el firmware y el reloj del ESP32 están bien y puedes empezar.

Paso 0.3. Dos datos que vas a necesitar

- **Tamaño de trama (F):** el mismo de la semana 6, es decir **128**. En todos los comandos de esta guía ya aparece `--frame 128`.
- **Velocidad del bus 3:** la necesitas en la semana 8 para el analizador. Consúltala con:

```
grep i2c3 /boot/firmware/config.txt
```

Si termina en `baudrate=100000` el bus va a **100 kbit/s**; si termina en `baudrate=400000`, a **400 kbit/s**. Anótalo.

Paso 0.4. Evitar que se corte una prueba larga (tmux)

Si la laptop se suspende o se cae el wifi, la conexión SSH se corta y la prueba que estaba corriendo se detiene. Para evitarlo, usa **tmux**, que mantiene la sesión viva en la Raspberry:

```
sudo apt install -y tmux        # solo la primera vez
tmux new -s lab                  # abre una sesión llamada lab
```

- Dentro de tmux trabajas igual que siempre (activa el entorno con el comando de 0.1).
- Para salir sin detener nada: **Ctrl+B**, suelta, y luego **D**.
- Para volver a entrar (aunque se haya cortado el SSH): `tmux attach -t lab`.

SEMANA 7 · TLS 1.3 híbrido sobre I2C

Objetivo	Establecer sesiones con el intercambio **híbrido X25519MLKEM768**, comprobar que de verdad se negocia el híbrido, revisar que al ESP32 le alcanza la memoria y hacer una primera comparación con el clásico.
Dedicación estimada	4–6 horas.
Antes de empezar	Semana 6 cerrada (la prueba del paso 0.2 da esp_estado 0).
Al terminar tendrán	Dos resultados guardados (clásico e híbrido), 30 corridas mezcladas correctas, la tabla de primera comparación en la bitácora y la etiqueta v0.7-hibrido en Git.

No hay que grabar nada en el ESP32: el mismo firmware de la semana 6 hace clásico e híbrido. La Raspberry le dice cuál usar en cada corrida.

Paso 1. Guardar una corrida clásica de referencia

```
cd ~/proyecto-kyber-rpi/pi && source ../src/venv/bin/activate
mkdir -p ../docs/evidencias
python3 servidor_tls.py --grupo clasico --frame 128 | tee ../docs/evidencias/s7_clasico.txt
```

`tee` muestra el resultado en pantalla y además lo guarda en el archivo. Revisa que `"esp_estado": 0` y anota `bytes_clienthello` (debe rondar **200**).

Paso 2. Primera sesión híbrida

```
cd ~/proyecto-kyber-rpi/pi && source ../src/venv/bin/activate
python3 servidor_tls.py --grupo hibrido --frame 128 | tee ../docs/evidencias/s7_hibrido.txt
```

Tarda un poco más que el clásico (hay más bytes que pasar por el bus). Al terminar, busca estos campos en el JSON:

Campo	Qué debe salir	Qué significa
grupo	hibrido	lo que pidió la Raspberry
grupo_detectado	hibrido	lo que realmente se negoció (se deduce del tamaño de la respuesta)
bytes_clienthello	≈ 1.380	primer mensaje del ESP32; incluye la clave pública ML-KEM (1.184 B)
bytes_vuelo_srv	≈ 1.800	respuesta de la Raspberry; incluye el texto cifrado ML-KEM (1.088 B)
esp_estado	0	el ESP32 terminó todo sin errores
error	no aparece	la Raspberry tampoco tuvo errores
esp_t_hs_us	un número > 0	duración del handshake medida en el ESP32 (microsegundos)
esp_heap_usado_hs	un número > 0	memoria que ocupó la sesión en el ESP32 (bytes)
esp_stack_libre	más de 2.000	pila que le sobró a la tarea TLS (bytes)

Compara con el archivo del paso 1: el ClientHello híbrido es unas **7 veces** más grande que el clásico. Esa diferencia es uno de los resultados centrales de la tesis.

Si algo sale distinto

Lo que ves	Causa probable	Qué hacer
esp_estado: -3	el reloj del ESP32 está en 1970	confirma que grabaste el main.c corregido de la semana 6
esp_estado: "sin_reporte" o TimeoutError solo en el híbrido	el ESP32 se quedó sin pila y se reinició	ve al paso 3 (subir la pila)
BAD_ECPOINT o esp_estado negativo solo en el híbrido	biblioteca wolfSSL antigua en el ESP32	debe ser 5.9.4 (semana 2); mándame el JSON completo
grupo_detectado: clasico pidiendo hibrido	no se negoció el híbrido	mándame el JSON completo

Paso 3. Memoria del ESP32 (solo si hace falta)

ML-KEM necesita más memoria que X25519. Si `esp_stack_libre` es **menor a 2.000**, o el híbrido da `"sin_reporte"`, hay que darle más pila a la tarea TLS:

- En la laptop abre `esp32/firmware/main/main.c` y busca (Ctrl+F) `"tls", 32768`. La línea completa es:

```
xTaskCreatePinnedToCore(tarea_tls, "tls", 32768, NULL, 5, NULL, 1);
```

- Cambia **32768** por **49152** y guarda.
- Pasa el USB del ESP32 a la laptop, abre la terminal de ESP-IDF, entra a la carpeta `esp32\firmware` y graba: `idf.py -p COM3 build flash` (con tu puerto).
- Regresa el USB a la Raspberry, espera 5 segundos y repite el paso 2.

Si `esp_stack_libre` ya era mayor a 2.000, **no cambies nada**. En los dos casos, anota en la bitácora `esp_stack_libre` y `esp_heap_usado_hs` del clásico y del híbrido: la diferencia de memoria es un resultado de la tesis.

Paso 4. Estabilidad: 30 corridas mezclando clásico e híbrido

Usa tmux (paso 0.4) porque esta prueba tarda varios minutos. Pega las dos primeras líneas juntas (la barra `\` continúa el comando en la línea siguiente):

```
cd ~/proyecto-kyber-rpi/pi && source ../src/venv/bin/activate
python3 campana.py --bloque prueba_hibrido --grupos clasico,hibrido --tamanos 128 \
                   --reps 15 --reposo 10 --pausa 1 --frame 128
python3 resumen_rapido.py | tee ../docs/evidencias/s7_resumen.txt
```

El script hace 15 corridas de cada grupo en orden aleatorio y mide la energía al mismo tiempo. Todo queda en `data/raw/`. La salida de `resumen_rapido.py` se lee así:

Línea de la salida	Qué debe decir
Corridas: 30 correctas: 30 con problema: 0	las 30 terminaron bien
Contadores del enlace (suma)	retransmisiones y errores en 0 o muy pocos
Medias por grupo	una fila clasico y una fila hibrido con 5 columnas (las de la tabla del paso 5)
Consistencia grupo pedido vs detectado: OK	nunca se negoció un grupo distinto al pedido

Si hay corridas con problema, la salida dice de qué tipo. Mándame el archivo `s7_resumen.txt` antes de seguir.

Paso 5. Tabla de primera comparación (en la bitácora)

Copia los valores de la sección **Medias por grupo** y calcula la diferencia:

Medida (media)	Clásico	Híbrido	Diferencia (%)
esp_t_hs_us (duración del handshake, µs)			
bytes_clienthello			
esp_heap_usado_hs (bytes)			
esp_stack_libre (bytes)			
rend_util_kbps (rendimiento útil)			

Diferencia (%) = (híbrido − clásico) ÷ clásico × 100. Ejemplo: si el clásico da 202 bytes y el híbrido 1.386, la diferencia es (1.386 − 202) ÷ 202 × 100 ≈ **586 %**.

> **Todavía no es el resultado de la tesis**
>
> Son solo 15 corridas por grupo y un tamaño de mensaje. Sirve para comprobar que los números tienen sentido. Los resultados oficiales salen de la campaña de las semanas 9 y 10.

Paso 6. Guardar el hito en Git

```
cd ~/proyecto-kyber-rpi
git add -A && git commit -m "Semana 7: TLS híbrido estable sobre I2C"
git tag -a v0.7-hibrido -m "Handshake híbrido estable sobre I2C"
git push && git push origin v0.7-hibrido
```

Si GitHub pide usuario y contraseña: el usuario es tu cuenta y la «contraseña» es un **token nuevo** (el que apareció en el PDF del repositorio debe estar revocado).

> **Criterio de cierre de la semana**
>
> - ☐ Corrida híbrida con grupo_detectado = hibrido y esp_estado = 0 (archivo s7_hibrido.txt).
> - ☐ ClientHello ≈ 1.380 B en el híbrido y ≈ 200 B en el clásico.
> - ☐ esp_stack_libre mayor a 2.000 en el híbrido, sin reinicios.
> - ☐ Resumen con 30 de 30 corridas correctas y consistencia OK (s7_resumen.txt).
> - ☐ Tabla de primera comparación en la bitácora y etiqueta v0.7-hibrido publicada.

SEMANA 8 · Confidencialidad e integridad (con el analizador lógico)

Objetivo	Demostrar con un instrumento independiente que, sin TLS, la telemetría se lee en el bus y, con TLS híbrido, no; y comprobar que TLS rechaza tramas alteradas o repetidas aunque el atacante recalcule el CRC.
Dedicación estimada	5–7 horas.
Antes de empezar	Semana 7 cerrada. Analizador lógico de 8 canales y 24 MHz, cables dupont macho-hembra.
Al terminar tendrán	Dos capturas del analizador (.sr) con sus imágenes, la verificación automática y 5 pruebas de ataque aprobadas.

Paso 1. Instalar PulseView en la laptop (una sola vez)

- Entra a la página de descargas de **sigrok** (busca «sigrok PulseView download»), descarga el instalador de **PulseView para Windows** (64 bits) e instálalo con las opciones por defecto.
- Conecta el analizador a un USB de la laptop.
- Abre **Zadig** desde el menú Inicio (se instala junto con PulseView). En la lista elige el analizador (puede llamarse *fx2lafw*, *Unknown Device* o *CY7C68013*; si no aparece, menú Options → List All Devices). A la derecha elige **WinUSB** y presiona **Install Driver** (o Replace Driver). Espera a que diga que terminó.
- Abre PulseView. En el selector de dispositivo de la barra superior debe aparecer **fx2lafw** (a veces dice «Saleae Logic» o «8 channels»). Si dice «Demo device», haz clic en él → **Connect to Device** → driver **fx2lafw** → **Scan for devices** → elige el que aparece → OK.

Paso 2. Conectar el analizador al bus (todo apagado)

Apaga la Raspberry (`sudo poweroff`, espera a que se apague la luz verde y desconecta la fuente). El analizador **solo escucha**: se conecta al bus del ESP32 (bus 3), **no al bus del INA219**.

Pin del analizador	Va a	Cómo
CH1 (o CH0)	fila SCL de la protoboard (la del pin 29 y G22)	punta macho del dupont en un agujero libre de esa fila
CH2 (o CH1)	fila SDA de la protoboard (la del pin 7 y G21)	punta macho del dupont en un agujero libre de esa fila
GND	un GND libre de la Raspberry (pin 14, 20, 25, 30, 34 o 39)	dupont hembra-hembra
USB	la laptop	—

```
Protoboard (lo que ya tienes + el analizador):
  fila SDA:  pin 7 — G21 — resistencia 1 — CH2 del analizador
  fila SCL:  pin 29 — G22 — resistencia 2 — CH1 del analizador
  GND del analizador — GND de la Raspberry
```

Tres cuidados

Sin el cable **GND** del analizador verás ruido o líneas planas: es obligatorio.

En PulseView los canales se llaman **D0, D1, D2…**: el pin rotulado CH1 es D0 y CH2 es D1 (si tu analizador rotula CH0, CH0 es D0). Anota cuál conectaste a SCL y cuál a SDA.

No conectes ningún pin del analizador a 5 V ni al pin 17 (3,3 V).

Enciende la Raspberry, entra por SSH (Ventana A) y corre la prueba del paso 0.2 para confirmar que el analizador conectado no estorba al bus.

Paso 3. Configurar PulseView

- **Frecuencia de muestreo** (lista junto al dispositivo, arriba): **2 MHz** si el bus va a 100 kbit/s; **4 MHz** si va a 400 kbit/s (dato del paso 0.3).
- **Número de muestras** (la lista de al lado): **50 M** a 2 MHz o **100 M** a 4 MHz. Eso da unos 25 segundos de grabación, suficiente para cada prueba.
- Deja visibles solo D0 y D1: clic en el ícono de canales (las ondas) y desmarca D2 a D7.
- Agrega el decodificador: botón **Add protocol decoder** (ícono amarillo y verde) → escribe **I2C** → doble clic en **I²C**.
- Haz clic en la etiqueta **I²C** que apareció a la izquierda y asigna **SCL = D0** y **SDA = D1** (según cómo conectaste en el paso 2).

Prueba rápida: presiona **Run** y, en la Ventana A, corre de nuevo la prueba del paso 0.2. Cuando PulseView se detenga, haz zoom (rueda del mouse) en la zona con actividad: debes ver pulsos en D0 y D1 y, debajo, el decodificador con cajitas de colores (Start, Address, Data, ACK, Stop). Si ves eso, el analizador está listo.

Paso 4. Captura de control: telemetría SIN TLS

- En PulseView presiona **Run**.
- Enseguida, en la Ventana A:

```
cd ~/proyecto-kyber-rpi/pi && source ../src/venv/bin/activate
python3 servidor_tls.py --grupo sin_tls --frame 128 --n 3 --bulk 0
```

- Espera a que PulseView se detenga solo (o presiona **Stop** cuando termine el comando).
- **Para ver el texto:** clic en la etiqueta I²C → en *Data format* elige **ascii**. Haz zoom en la actividad hasta leer **TELEMETRIA|run=…|i=0|…** letra por letra en las cajitas *Data write/read*.
- **Toma una captura de pantalla** donde se lea (Windows + Shift + S) y guárdala como `captura_sin_tls.png`.
- **Guarda la captura del analizador:** menú *File* → *Save As* → `captura_sin_tls.sr`.
- **Exporta las anotaciones:** vuelve a poner *Data format* en **hex** (importante: el script de verificación lee números hexadecimales). Luego clic derecho sobre la fila del decodificador → **Export all annotations** → `anotaciones_sin_tls.txt`.

Guarda los tres archivos en una carpeta de la laptop, por ejemplo `Documentos\semana8`.

Paso 5. Captura con TLS híbrido

Repite exactamente el paso 4, pero con este comando y estos nombres de archivo:

```
python3 servidor_tls.py --grupo hibrido --frame 128 --n 3 --bulk 0
```

`captura_hibrido.png`, `captura_hibrido.sr` y `anotaciones_hibrido.txt`. Esta vez, con el formato en ascii, **no debe leerse «TELEMETRIA» en ningún lado**: solo caracteres sin sentido, porque los datos viajan cifrados. La captura dura más que la anterior (hay un handshake de por medio).

Paso 6. Pasar los archivos a la Raspberry

En una PowerShell **de la laptop** (sin entrar por SSH), ve a la carpeta donde guardaste todo y copia los archivos con `scp` (pide la contraseña de la Raspberry):

```
cd $HOME\Documents\semana8
$d = "usuariok@integradoraPi4.local:~/proyecto-kyber-rpi/docs/evidencias/"
scp anotaciones_sin_tls.txt anotaciones_hibrido.txt $d
scp captura_sin_tls.sr captura_hibrido.sr captura_sin_tls.png captura_hibrido.png $d
```

Paso 7. Verificación automática

En la Ventana A (Raspberry):

```
cd ~/proyecto-kyber-rpi
python3 herramientas/buscar_marcador_analizador.py docs/evidencias/anotaciones_sin_tls.txt
python3 herramientas/buscar_marcador_analizador.py docs/evidencias/anotaciones_hibrido.txt
```

Archivo	Resultado esperado
anotaciones_sin_tls.txt	Bytes decodificados mayor a 0 y **MARCADOR VISIBLE en el bus** (en al menos un sentido)
anotaciones_hibrido.txt	Bytes decodificados mayor a 0 y **marcador NO encontrado** en los dos sentidos

Si dice **Bytes decodificados: 0**, exportaste con el formato en ascii: vuelve a ponerlo en **hex** en PulseView y exporta de nuevo.

Paso 8. Pruebas de manipulación y repetición (no usan el analizador)

`ataque.py` simula a un atacante conectado al bus: cambia un bit de un mensaje cifrado (en los dos sentidos) o repite una trama ya entregada, y recalcula el CRC para que la capa de enlace no lo note. Se repite 5 veces para mostrar que el resultado es estable:

```
cd ~/proyecto-kyber-rpi/pi && source ../src/venv/bin/activate
for i in 1 2 3 4 5; do python3 ataque.py --frame 128 | grep CONCLUSION; done
```

Las 5 veces debe salir `"CONCLUSION": "TODAS LAS PRUEBAS PASARON"`. Cada ejecución guarda su detalle en `data/raw/ataques_*.json`. Qué comprueba cada prueba:

Prueba	Qué hace el atacante	Resultado esperado
claro_sin_tls	lee el bus sin TLS	el texto **se ve**
claro_con_tls	lee el bus con TLS híbrido	el texto **no se ve**
alterar_esp_a_pi	cambia un bit de un mensaje del ESP32	la Raspberry lo **rechaza**
alterar_pi_a_esp	cambia un bit de un mensaje de la Raspberry	el ESP32 lo **rechaza**
reinyectar	repite una trama vieja	se **rechaza**

Si alguna vez sale «REVISAR», corre una vez sin el `grep` (`python3 ataque.py --frame 128`) y mándame la salida completa.

Paso 9. Interpretación (copiar en la bitácora)

Párrafo para el Capítulo 3

El CRC de la capa de enlace protege contra errores accidentales del bus, pero un atacante puede recalcularlo. TLS 1.3 usa cifrado autenticado (AEAD) con un número de secuencia por registro, por lo que cualquier alteración o repetición invalida la etiqueta de autenticación y la sesión se aborta. Estas pruebas verifican el comportamiento de la implementación; no constituyen una prueba formal de seguridad, y la resistencia frente a una computadora cuántica proviene del diseño del estándar, no de este experimento.

Paso 10. Commit y push

```
cd ~/proyecto-kyber-rpi
git add -A && git commit -m "Semana 8: confidencialidad con analizador y pruebas de ataque"
git push
```

Criterio de cierre de la semana

- ☐ captura_sin_tls.sr y captura_hibrido.sr guardadas, con sus imágenes .png.
- ☐ buscar_marcador_analizador.py: VISIBLE sin TLS y NO encontrado con TLS.
- ☐ ataque.py: TODAS LAS PRUEBAS PASARON las 5 veces (JSON en data/raw/).
- ☐ Párrafo de interpretación en la bitácora.
- ☐ Commit y push hechos.

Problemas comunes con el analizador

Lo que ves	Solución
PulseView solo muestra «Demo device»	falta el driver: repite Zadig (paso 1) y reconecta
Líneas planas en D0 y D1	falta el GND del analizador o los cables están en otra fila
El decodificador no muestra nada o muestra errores	SCL y SDA invertidos en la configuración del decodificador, o frecuencia de muestreo baja
La captura termina antes de que acabe el comando	sube el número de muestras
El comando de la Raspberry falla con el analizador conectado	revisa que ningún cable del analizador toque 3,3 V o 5 V